

P1303R0

Inline Module Partitions

Published Proposal, 2018-10-07

This version:

<https://wg21.link/p1303>

Author:

[Isabella Muerte](#)

Audience:

EWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Current Render:

[P1303](#)

Current Source:

[slurps-mad-rips/papers/proposals/inline-modules.bs](https://ericniebler.com/2018/10/07/slurps-mad-rips/papers/proposals/inline-modules.bs)

Abstract

Table of Contents

1 Revision History

1.1 Revision 0

2 Motivation

3 Design

4 Examples

4.1 Multi source

References

Informative References

§ 1. Revision History

§ 1.1. Revision 0

Initial Release 🎉

§ 2. Motivation

This paper is an addendum to the [\[P1242\]](#) proposal. That paper proposes having a module consist of *at most* two parts: An interface and implementation part. These are separated by some syntactic marker, (in this case `module :private` as `private` is a reserved identifier). However, this proposal goes a step further and advocates having a nearly unlimited number of so-called *inline module partitions* per file. The author of this paper argues that [\[P1242\]](#) should be the minimum interface implemented for C++20, but that an alternative superset is available and that's what this proposal discusses.

§ 3. Design

[\[P1242\]](#) states that

If a module interface unit includes an inline module implementation partition, it will appear after the interface itself and be separated from the interface by some syntactic divider.

The current syntax it proposes in its examples is `module :private`. This paper instead relies on the `inline` keyword and any desired identifier for a module partition. So the equivalent use would be `inline module :impl`.

The reason we don't reserve an identifier for this *partition marker* is to permit users to define what partition naming conventions make most sense to them. This approach might not have been possible before C++20, however the precedence of Contracts requiring vendors to provide a compiler flag gives us the same capabilities if desired. In this case, we can compile a module interface unit *up to* a given *marker*. This also provides us with one additional feature regarding code. Users can now theoretically place their unit tests under an inline module partition, and compile the file twice, once for the actual

code to be compiled into an executing program, and again to include unit tests, without having to bloat the final executable or library.

Thus, the structure of a module interface unit will be as follows:

1. Module interface partition
2. [N](#) Optional inline module partitions

When a module interface partition is imported, its corresponding inline module implementation partitions are imported up to the same *partition marker* as the current primary module interface partition.

Much like [\[P1242\]](#), inline module partitions are never re-exported.

§ 4. Examples

These examples are adapted from [\[P1242\]](#)

```
export module m;
struct s;
export usig s_ptr = s*;

inline module :impl;
struct s { };
```

§ 4.1. Multi source

Module interface unit for partition `m:impl`:

```
export module m :impl;
struct s;
s* f ();

inline module :priv;
struct s { };
```

Primary module interface unit for `m`:

```
export module m;  
export import m:impl;
```

Translation unit using `m`:

```
import m;  
s var; // ill-formed, S is incomplete  
s* s_ptr = f(); // OK  
auto sz = sizeof(*f()); // ill-formed, S is incomplete
```

§ References

§ Informative References

[P1242]

Jeff Snyder. [Single-file modules with the Atom semantic properties rule](https://wg21.link/p1242r0). URL:
<https://wg21.link/p1242r0>